# ExCommand

## Iacob Eduard-Stefan si Manolache Dumitru-Alexandru

## Grupa 1207B

**Gen:**2D,Action-Platformer,Sci-Fi
**Platformă:** Windows/Linux (Java)

### Scurtă descriere :

exCommand te pune în pielea unui comandant experimentat al unei Flote Spațiale aflate într-o misiune de rutină, transformată brusc într-un coșmar. După un atac devastator al unei rase necunoscute de extratereștri, comandantul se trezește dezorientat în urma unei explozii puternice care a avariat grav nava. Acesta investigheaza si presupune este singurul supraviețuitor, neexistand urma de echipajul la care era in frunte. Pe o planeta plină de pericole și inamici care patrulează fiecare colț, comandantul trebuie să lupte pentru a recăpăta controlul și a-și răzbuna echipajul. Folosind armele din dotare si armura ExoSuit, acesta pornește într-o misiune disperată de supraviețuire și răzbunare, înfruntând amenințarea extraterestră și încercând să scape cu viață.

---

## 2. Lista Sarcinilor Săptămânale

**Echipă (2 membri):**

| Săptămâna | Sarcini |
|---|---|
| 8 | Incarcarea hartii nivel 1 (Manolache) + meniu principal (Iacob) |
| 9 | Implementare deplasare personaj (Manolache)+ camera (Iacob) |
| 10 | Tratarea Coliziunilor (Amandoi) |
| 11 | Alte nivele + inamici + NPC (Amandoi) |
| 12 | Salvarea si incarcarea jocului dintr-o baza de date<br>Implementare baza de date (Manolache) + Salvare si incarcare (Iacob) |
| 13 | Finisare (Amandoi) |
| 14 | Prezentare (Amandoi) |

---

## 3. Proiectarea Sistemului

- **New Game:** jocul incepe cu caracterul principal care are toate abilitatile deblocate cu cooldown crescut si cu damage mic (in cazul abilitatilor de atac)
- **Progress:** pe parcursul calatoriei, jucatorul intalneste inamici noi (Zax, ExoStar) si provocari noi (BulletHell level, parkour phase la nivelul 3). Pentru fiecare inamic invins, jucatorul primeste 100 de puncte si pierde puncte din score atunci cand isi ia damage. In caz de scor egal, departajarea se va tinand cont de un TimeScore.

  Inainte de a trece la un nou nivel, jucatorul este obligat sa elimine toti inamicii din nivelul Curent. Un caz special este la nivelul 2 unde jucatorul trebuie sa se fereasca timp de 2 minute.
- **Atacuri Erou:** basic attack (click), double attack (abilitate speciala), atac bidirectional (abilitate speciala), pierce shot (abilitate speciala). Eroul va avea doar atacuri ranged.
- **Atacuri Inamici:**

  Nivel 1: damage scazut in cazul inamicilor mici (de tip melee sau ranged) si damage+health mai ridicat in cazul MiniBossului.

  Nivel 2 (Bullet-time event): jucatorul isi va lua one shot la orice tip de contact cu laserul ExoStar-ului.

  Nivel 3: damage crescut semnificativ si la monstrii mici cat si la Bossul final.
- **Input Handling:**
- Tastatură pentru mișcare (WASD), abilități (SHIFT pentru dash, E pentru health, F pentru Interactiunea cu NPCul).
- Mouse pentru camera si atacuri normale.
- **Damage System:** Inamici aplică damage între 10-15 HP random, acest interval fiind ajustat in functie de nivel (pe baza avansarii in joc acesta creste).
- **Win!:** cand jucatorul invinge bossul final acesta castiga jocul, scorul fiind calculat in functie de damageul luat pe parcursul runului si de numarul de cristale ramase.

## Mecanici cheie:

- **Sistem de viață (HP)** și damage dinamic (inamici lovesc cu damage random pe un anumit interval).
- **Abilități speciale:** Dash, Bullet-Time (evenimente de evitare proiectile).
- **NPC interactivi:** Pinguinul Linux ce are rol de a oferi informatii jucatorului in legatura cu scopul jocului
  • **Tranziții între niveluri:** Portaluri după înfrângerea tuturor inamicilor si colectarea KeyCardului.

## 4. Proiectarea Conținutului

**Sprite-uri:**

1. **Comandantul** (ExoSuit):
    - Animatii: mers, dash, parry, atac bidirectional.
    - Atacuri: atac normal, doublu atac, atac bidirectional, pierce shot.
    - Abilitate defenisva: parry/block.
2. **IceGolem** (Minibossul de la nivelul 1)
3. **Zax** (Boss final)
4. **Imp** (inamic dificultate medie de la nivelul 1)
5. **ExoStar** (inamicul care trage cu lasere de la nivelul 2)

**Mediu:**

- **ExoPlaneta:** CrashSite (nivel 1), SecurityRoom (nivel 2), Laborator (boss final).
- **Efecte:** Explozii, fum, lumină intermitentă, laser.
- **Background:** Imagini reprezentative pentru fiecare nivel.

**Sunet:**

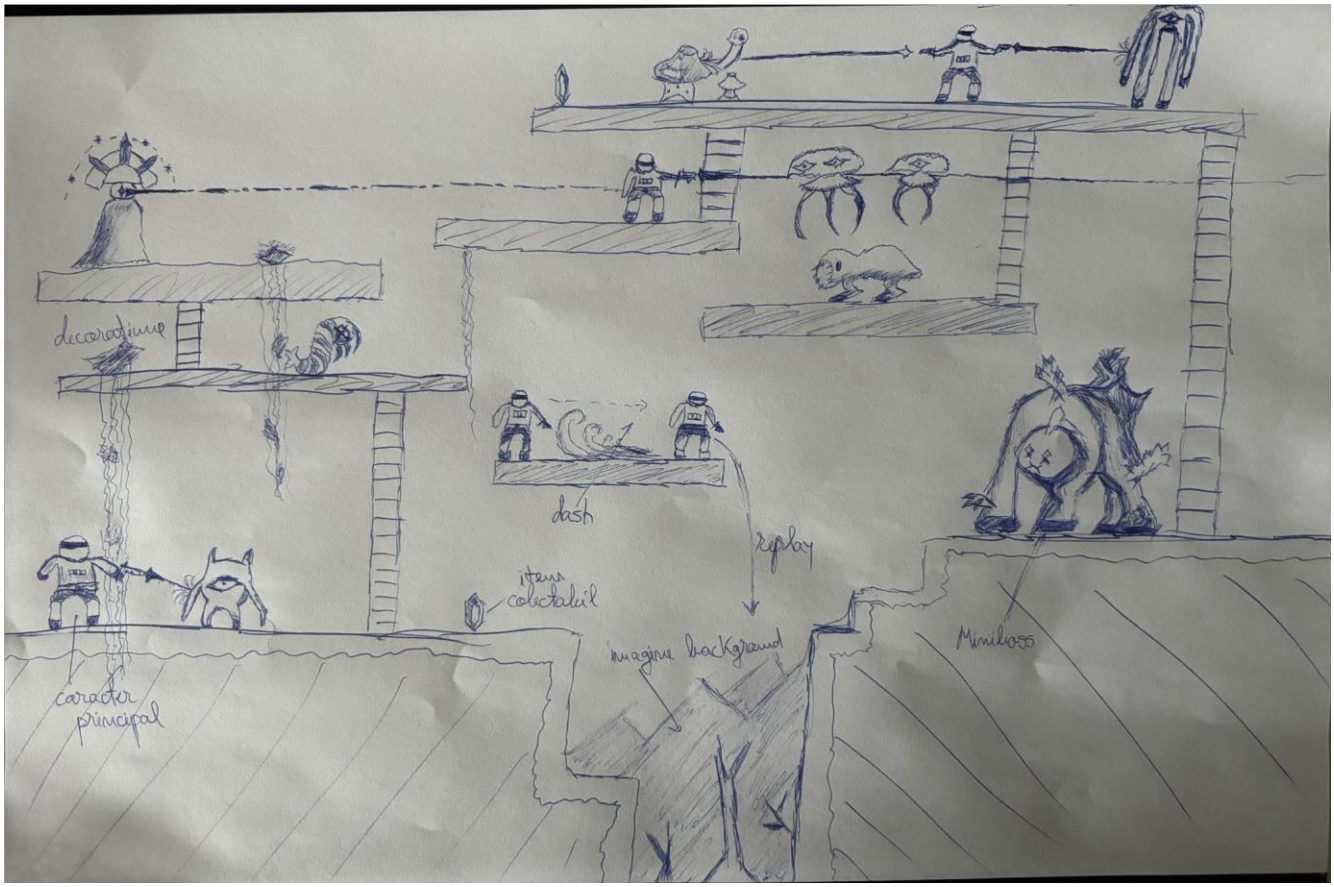
- Muzică tensionantă, efecte de lovituri, muzica techno.

---

## 5. Proiectarea Nivelurilor (3 niveluri)

| Nivel | Descriere | Dificultate |
|---|---|---|
| **1: Trezirea** | Dungeon Progression, cu inamici | Ușor (inamici slabi) |
| **2: Security Room** | Bullet-Hell Level | Mediu (proiectile) |
| **3: Laborator** | Inamici mai puternici + Final Boss | Greu (AI agresiv) |

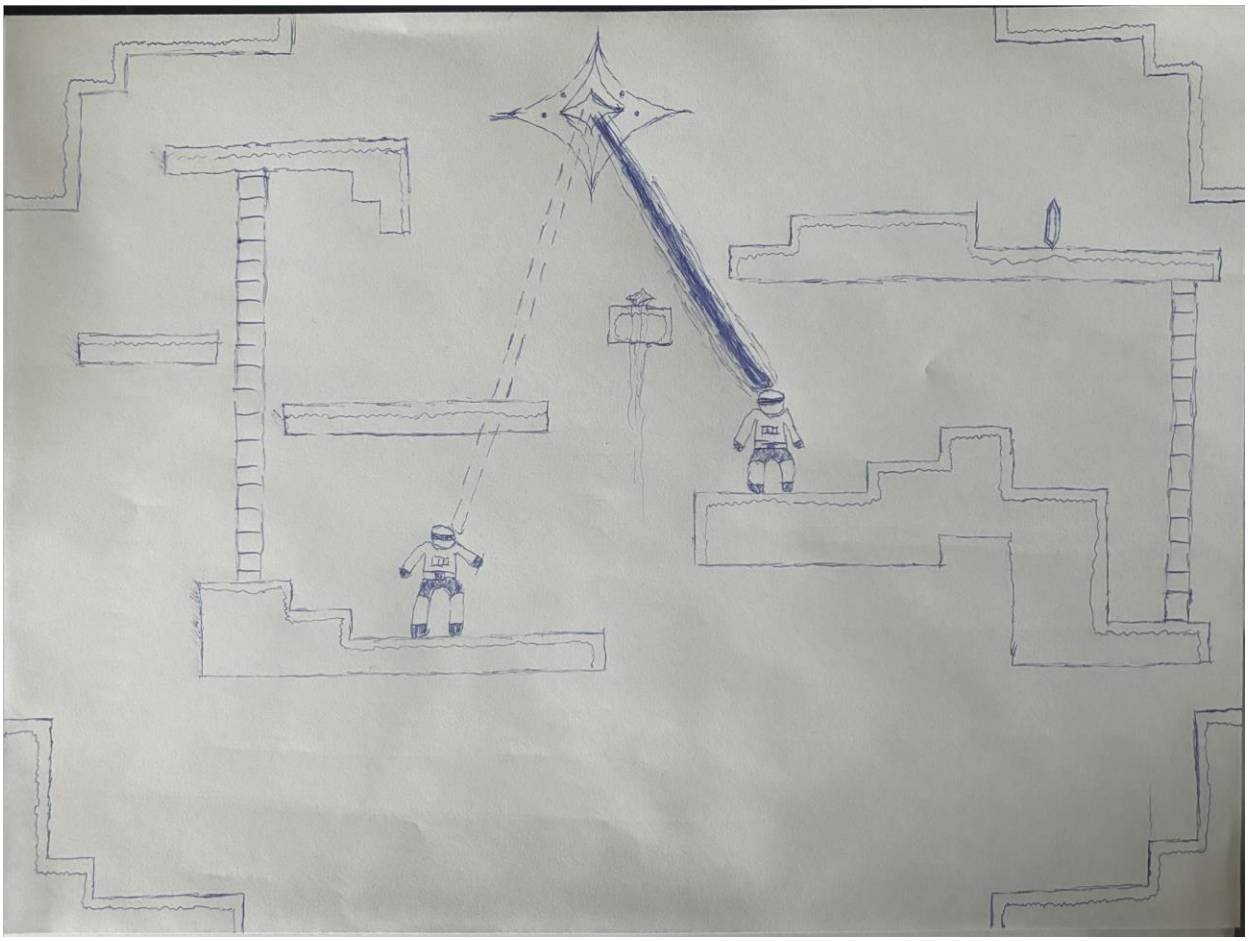
---



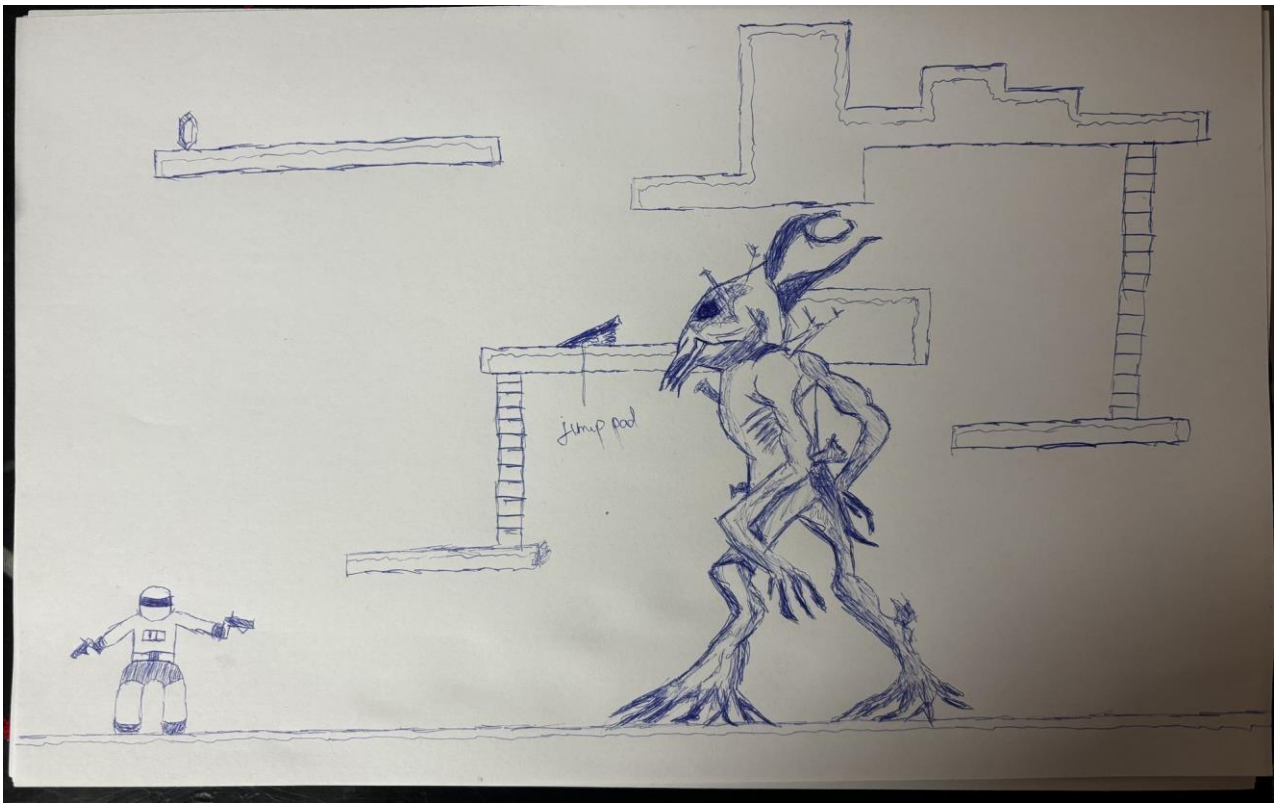

â

In poza este prezentat primul nivel al jocului. Eroul se va putea misca cu tastele W, A, S, D (W si S pentru scara), SPACE pentru jump, SHIFT pentru dash. De asemenea in schita sunt prezentate si atacurile protagonistului: Stanga jos avem atacul simplu, in mijloc avem mecanica de dash, mai sus avem atacul de tip pierce shot si cel mai sus in desen este prezentat atacul bidirectional.

Pentru a putea avansa la urmatorul nivel, cel de bullet-hell, jucatorul trebuie sa invinga Minibossul din partea dreapta a desenului.



In aceasta poza este prezentat nivelul special al jocului, cel de bullet-hell. Jucatorul are datoria de a se feri de toate laserele ExoStar-ului. Acesta va genera o urma a laserului (raza din partea stanga) inainte de a da damage cu raza propriu-zisa (raza din partea dreapta), pentru a da ocazia caracterului sa se fereasca. Dupa un timp, atacurile se vor opri si daca personajul principal a ramas in viata, acesta va putea avansa la urmatorul nivel. In timpul nivelului, jucatorul poate colecta cristale pentru a putea cumpara iteme ce ii vor folosi mai tarziu in luptele din ultimul nivel.




In poza de mai sus este prezentata o schita a ultimului nivel. Acest nivel va fi compus dintr o parte care contine inamici la fel ca in primul nivel, doar ca acestia vor avea o dificultate mai mare, si o parte cu bossul final (cel din imagine). Boss ul final va avea cea mai ridicata dificultate din joc in maniera de AI, cu posibilitate de atac melee sau ranged (atac cu gheara sau cu proiectile). Odata ce jucatorul invinge bossul final acesta va termina jocul. Scorul jucatorului va fi stabilit de currencyul ramas si dupa damageul luat pe parcursul jocului. Scorul maxim pentru damageul luat este Perfect Run cu 1000 de puncte. In caz ca se ia damage, din scorul acesta vor fi scazute 50 de puncte/hit.

## 6. Interfața Utilizatorului (UI)

- **HUD:** HP bar, Hear bar, cooldown abilități, iteme care pot fi folosite.
- **Meniu principal:** Start, Load, Settings (opțiuni audio/pauză).
- **Ecran de tutorial:** Imagine cu controale (WASD, SHIFT, Q).

---

**7. Resurse Bibliografice**

- **Sprite-uri:** OpenGameArt.
- **Game Design:** "Game Programming Patterns" (Robert Nystrom).
- **Java:** Documentația oficială Oracle pentru AWT/Swing.

**8.** Progresul Saptamanal si coduri relevante

❖ Incarcarea hartilor pentru nivel :

➢ Realizarea hartii a fost facuta prin intermediul instatierii de obiecte corespuzatoare fiecarui tip de tile cu un id unic ca mai apoi fiind desenate si redesenate pentru optimizarea jocului. Acesta matrice provine dintr-un fisier .XML provenind de la un software specializat pentru editare de harti bazate pe tile-uri

pentru creearea si editarea mai usoara a hartilor Map1, Map2 si Map3.

```java
package PaooGame.Tiles;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import java.io.File;
///Clasa XmlToMatrix este responsabilă pentru citirea unui fișier XML care conține date despre un layer de tile-uri și transformarea acestor
/// Această matrice poate fi utilizată ulterior pentru a construi o hartă a jocului, unde fiecare ID corespunde unui tip specific de tile.
/// Aceasta clasa este pentru usurarea editarii hartilor ,deoarece am realizat-o intr-un editor grafic special numit Tiled.
public class XmlToMatrix {

    public static int [][] GetIdMatrix(File xmlFile,int height, int width) {
        try {
            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
            Document doc = dBuilder.parse(xmlFile);
            doc.getDocumentElement().normalize();

            Node layerNode = doc.getElementsByTagName("layer").item(0); // Get the first layer
            if (layerNode.getNodeType() == Node.ELEMENT_NODE) {
                Element layerElement = (Element) layerNode;
                String csvData = layerElement.getElementsByTagName("data").item(0).getTextContent();
                String[] rows = csvData.split("\n");
                int[][] matrix = new int[rows.length][];
                for (int i = 0; i < rows.length; i++) {
                    String[] elements = rows[i].split(",");
                    matrix[i] = new int[elements.length];
                    for (int j = 0; j < elements.length; j++) {
                        String element = elements[j].trim(); // Trim whitespace
                        if (!element.isEmpty()) { // Check if the string is not empty
                            try {
                                matrix[i][j] = Integer.parseInt(element);
                            } catch (NumberFormatException e) {
                                System.err.println("Error parsing element: " + element);
                                matrix[i][j] = 0; // Or handle the error as needed
                            }
                        } else {
                            matrix[i][j] = 0; // Handle empty string case
                        }
                    }
                }
                int[][] matRet = new int[height][width];
                //int[][] matRet2 = new int[34][70];
                for(int i = 0; i <height; i++)
                {
                    for(int j = 0; j < width; j++)
                    {
                        matRet[i][j] = matrix[i+1][j];
                    }
                }
                return matRet;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

**Aceasta este mai apoi alocata unor clase speciale de gestionare a hartilor pentru a asigura ca toate matricile pentru marti sunt stocate si mai apoi folosite corect.**

```java
public Tile[][] getTileMap(){
    return tileMap;
}

public void Draw(Graphics g) {
    for (int y = 0; y < tileMap.length; y++) {
        for (int x = 0; x < tileMap[0].length; x++) {
            if (tileMap[y][x] != null) {
                tileMap[y][x].Draw(g, x * Tile.TILE_WIDTH, y * Tile.TILE_HEIGHT);
            }
        }
    }
}
```

- Desenarea entitatilor si selectarea starilor

- Acestea li se atribuie cate o stare pentru a selecta si animatia acestora si comportamentrul. Fiecare stare are cate un sprite diferite pentru realizarea animatiilor . de asemenea implementarea tastaturii si mouse-lui pentru actiunile jucatorului

```
public void update(boolean[] keys) {
        updatePrevPosition();
        moving = false;
        shooting = false;

        if (keys[KeyEvent.VK_Q]) {
            activateSuppressiveFire();
        }
        if (keys[KeyEvent.VK_E] && !ePressedLastFrame) {
            useImunosupressor();
            ePressedLastFrame = true;
        } else if (!keys[KeyEvent.VK_E]) {
            ePressedLastFrame = false;
        }

        if (metalJacketCooldown > 0) {
            metalJacketCooldown--;
        }

        if (metalJacketActive) {
            metalJacketTimer++;
            if (metalJacketTimer >= metalJacketDuration) {
                metalJacketActive = false;
                metalJacketTimer = 0;
            }
        }
        if (suppressing) {
            suppressTimer--;
            suppressTick++;

            if (suppressTick % suppressFireRate == 0) {
                if (suppressFireRight) {
                    audioPlayer.playSoundEffect("/sounds/gunshot.wav");
                    bullets.add(new Bullet(x + 15, y + 15, true, Assets.bullet, Assets.bullet_trace));
                } else {
                    audioPlayer.playSoundEffect("/sounds/gunshot.wav");
                    bullets.add(new Bullet(x - 10, y + 15, false, Assets.bullet, Assets.bullet_trace));
                }
                suppressFireRight = !suppressFireRight;
            }

            if (suppressTimer <= 0) {
                suppressing = false;
                suppressTick = 0;
                suppressFireRight = true;
            }

        } else {
            if (keys[KeyEvent.VK_A]) {
                x -= (int) speedX;
                moving = true;
                facingRight = false;
            }
            if (keys[KeyEvent.VK_D]) {
                x += (int) speedX;
                moving = true;
                facingRight = true;
            }
            if (keys[KeyEvent.VK_SPACE]) {
                jump();
            }
            dash(keys);
```

**De asmenea am implementat pentru background si functia de parallax pentru tranzitia usoara a elementor din spate(Luna Debian si nori ) pentru un visual mai placuta jucatorului.**

- Meniul principal contine functiile de NewGame, Settings, ExitGame , Load Game si Save Game, aceste toate fiind functionale. Cu ajutorul acestora poti incepe un joc nou ,modificarea sunetului in joc, salvarea progresului si incarcarea acestuia mai tarziu in cazul in care jucatorul ajunge pana intr-un punct si vrea sa revina la acel save.

```
public void selectOption() {
    switch (selectedOption) {
        case 0: // New Game
            gameStarted = true;
            isVisible = false;
            break;
        case 1: // Save Game
            System.out.println("Saving game...");
            GameState saveState = new GameState(
                "Player1",
                Game.score,
                (int) ((System.currentTimeMillis() - Game.startTime) / 1000),
                "LEVEL" + game.getCurrentMap(),
                Game.getHealth(),
                true,
                Game.player.getX(),
                Game.player.getY(),
                game.getCurrentMap(),
                Game.getEnemies1(), // Lista de inamici pentru map1
                Game.getEnemies3()  // Lista de inamici pentru map3
            );
            DatabaseManager.saveGame(saveState); // Salvăm starea jocului în baza de date
            break;

        case 2: // Load Game
            System.out.println("Loading game...");
            GameState loadState = DatabaseManager.loadLastGame(); // Încărcăm starea jocului din b
            if (loadState != null) {
                // Restaurăm lista de inamici încărcată
                Game.setEnemies1(loadState.getEnemy1());
                Game.setEnemies3(loadState.getEnemy3());

                // Restaurăm și alte stări necesare
                Game.score = loadState.getScore();
                Game.player.setPosition(loadState.getPlayerX(), loadState.getPlayerY());
                Game.player.setHp(loadState.getHealth());
                game.setCurrentMap(loadState.getCurrentMap());

                // Actualizăm timpul rămas
                Game.startTime = System.currentTimeMillis() - (loadState.getTimeSpent() * 1000L);
                gameStarted = true;
                isVisible = false;
            }
            break;



        case 3: // Settings
            fromMainMenu = true;
            isVisible = false;
            if (settingsMenu != null) {
                settingsMenu.setFromMainMenu(true);
                settingsMenu.toggle();
            }
            break;
        case 4: // Exit
            System.exit(0);
            break;
```

- Camera este implementata prin urmarirea jucatorului si centrata pe el, oprirea acesteia in limitele hartii deoarece dimensiunile ecranului depasesc dimesiunile monitorului si este nevoie de deplasarea acesteia

pentru a fii posibila parcurgerea jocului. Fiecare nivel are cate o camera proprie alocata, nivele fiind de diferite dimesiuni

```
import PaooGame.Entities.Player;
import java.awt.*;

///Clasa Camera este responsabilă pentru gestionarea poziției și transformării camerei în joc.
/// Aceasta permite centrul camerei să urmeze jucătorul și să limiteze vizibilitatea la dimensiunile hărții.
/// /// Camera poate fi utilizată pentru a începe și termina redarea graficii, asigurându-se că totul este desenat corect în raport cu poziția cam
public class Camera {
    private float x, y;
    private Player player;
    private int screenWidth, screenHeight;
    private int mapWidth, mapHeight;
    private Graphics2D originalGraphics;

    public Camera(Player player, int screenWidth, int screenHeight, int mapWidth, int mapHeight) {
        this.player = player;
        this.screenWidth = screenWidth;
        this.screenHeight = screenHeight;
        this.mapWidth = mapWidth;
        this.mapHeight = mapHeight;
    }

    public void update() {
        // Centrăm camera pe jucător
        x = player.getX() - screenWidth/2 + player.getWidth()/2;
        y = player.getY() - screenHeight/2 + player.getHeight()/2;

        // Limităm camera la dimensiunile hărții
        x = Math.max(0, Math.min(x, mapWidth - screenWidth));
        y = Math.max(0, Math.min(y, mapHeight - screenHeight));
    }


    public void beginRender(Graphics g) {
        originalGraphics = (Graphics2D) ((Graphics2D) g).create();
        ((Graphics2D) g).translate(-x, -y);
    }

    public void endRender(Graphics g) {
        if (originalGraphics != null) {
            ((Graphics2D) g).setTransform(originalGraphics.getTransform());
            originalGraphics.dispose();
        }
    }
```

- Coliziunile sunt implementate pentru toate entitatile din joc, player, inamici, bullet-uri si colectabile iar pentru fiecare este facut cate o  metoda si toate acestea sunt administrate de CollisionManager, fiecare de asemenea avand initializate un set de coliziuni in functie de nivel. La toate entitatile preia cate un dreptunghi in care verifica colturile acestuia cu matricea de tile-uri.

```java
public class CollisionManager { 6 usages ManolacheAlexandru54 +2
    private final Tile[][] tileMap; 25 usages
    public CollisionManager(Tile[][] tileMap) { this.tileMap = tileMap; }

    public void checkEntityCollision(MovableEntity entity) {...}

    private void checkVertical(MovableEntity player) {...}



    private void checkHorizontal(MovableEntity player) {...}


    public void checkBulletCollisions(Bullet bullet) {...}

    public void checkBulletHitsEnemies(List<Enemy> enemies, Bullet bullet) {...}

    public void checkBulletHitsPlayer(Player player, List<Bullet> bullets) {...}


    public void checkEntityHazardCollision(Player player) {...}
    public void checkImunosupressorCollision(Player player, List<Imunosupressor> items) {...
    public void checkKeyCardCollision(Player player, KeyCard keyCard) {...}


    public void checkPlayerEnemyCollision(Player player, ArrayList<Enemy> enemies) {...}
```

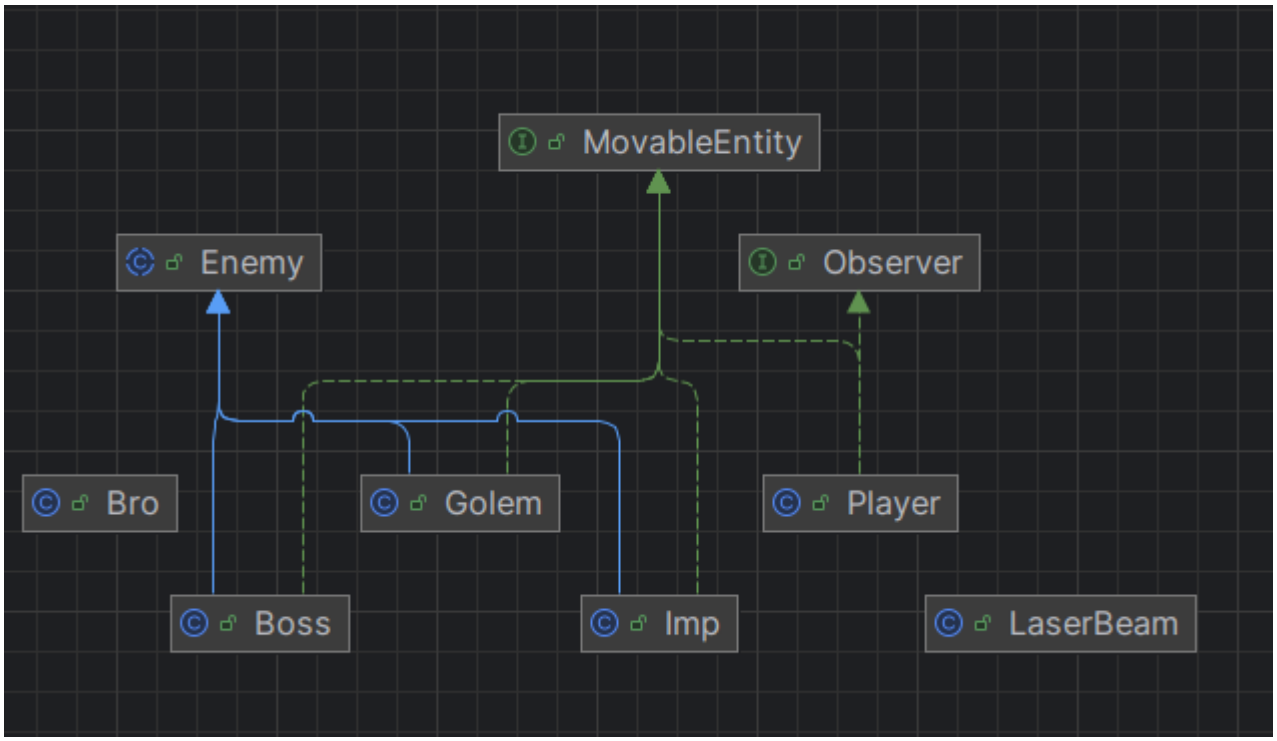
- **Baza de date este administrat de DataBaseManager in care se stocheaza tot ce este necesar pentru funtionalitatea NewGame, Load-ului si Save-ului, cum arf ii coordonate, cate charge-uri de imunosupresoare sunt  colectate , viata, numarul de inamici, etc,**

```java
public static void saveGame(GameState gameState) {
    String query = """
        INSERT INTO game_save
        (player_name, score, timeScore, level, health, game_status, player_x, player_y, current_map, enemies
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """;

    try (Connection conn = DriverManager.getConnection(DB_URL);
         PreparedStatement pstmt = conn.prepareStatement(query)) {

        Gson gson = new Gson();

        // Salvează inamicii
        ArrayList<String> enemiesMap1JsonList = Enemy.convertEnemiesToData(Game.getEnemies1());
        ArrayList<String> enemiesMap3JsonList = Enemy.convertEnemiesToData(Game.getEnemies3());
        String enemiesMap1Json = gson.toJson(enemiesMap1JsonList);
        String enemiesMap3Json = gson.toJson(enemiesMap3JsonList);

        // Salvează KeyCard
        String keycardState = gson.toJson(Game.keyCard.isCollected()); // true dacă a fost colectat

        // Salvează Imunosupressors corect ca liste de obiecte JSON
        ArrayList<Map<String, Object>> imunosData1 = new ArrayList<>();
        for (Imunosupressor imunos : Game.imunosupressorsMap1) {
            Map<String, Object> imunosData = new HashMap<>();
            imunosData.put("x", imunos.getX());
            imunosData.put("y", imunos.getY());
            imunosData.put("collected", imunos.isCollected());
            imunosData1.add(imunosData);
        }
        String imunosMap1Json = gson.toJson(imunosData1);

        ArrayList<Map<String, Object>> imunosData3 = new ArrayList<>();
        for (Imunosupressor imunos : Game.imunosupressorsMap3) {
            Map<String, Object> imunosData = new HashMap<>();
            imunosData.put("x", imunos.getX());
            imunosData.put("y", imunos.getY());
            imunosData.put("collected", imunos.isCollected());
            imunosData3.add(imunosData);
        }
        String imunosMap3Json = gson.toJson(imunosData3);

        // Setăm datele pentru fiecare coloană
        pstmt.setString(1, gameState.getPlayerName());
        pstmt.setInt(2, gameState.getScore());
        pstmt.setInt(3, gameState.getTimeSpent());
        pstmt.setString(4, gameState.getLevel());
        pstmt.setInt(5, gameState.getHealth());
        pstmt.setBoolean(6, gameState.isGameStatus());
        pstmt.setDouble(7, gameState.getPlayerX());
        pstmt.setDouble(8, gameState.getPlayerY());
        pstmt.setInt(9, gameState.getCurrentMap());
        pstmt.setString(10, enemiesMap1Json);
        pstmt.setString(11, enemiesMap3Json);
        pstmt.setString(12, keycardState);
        pstmt.setString(13, imunosMap1Json);
        pstmt.setString(14, imunosMap3Json);
        pstmt.setInt(15, Game.player.Imunosupressor_Count);
        pstmt.setInt(16, Game.keyCard.getX()); // Coordonata X a keyCard-ului
        pstmt.setInt(17, Game.keyCard.getY()); // Coordonata Y a keyCard-ului


        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.out.println("Error saving game state: " + e.getMessage());
    }
```

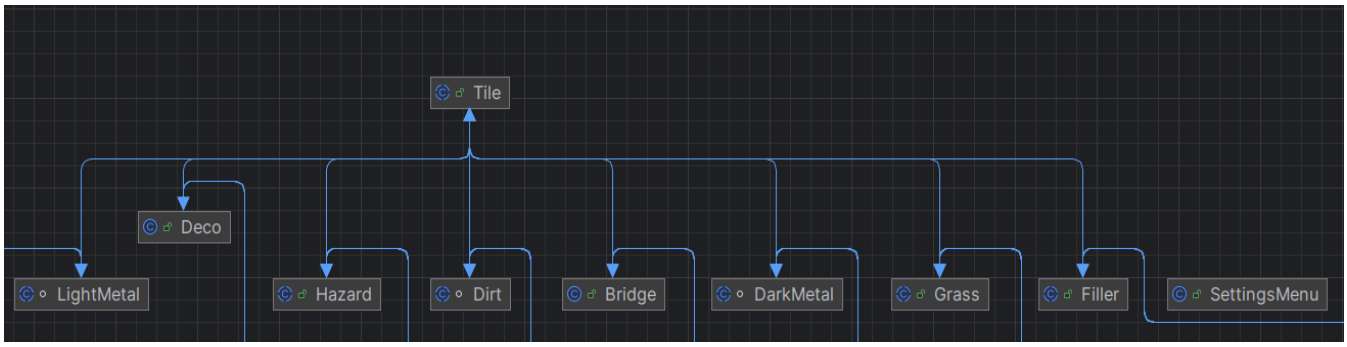
```java
    public static GameState loadLastGame() {
        String query = "SELECT * FROM game_save ORDER BY id DESC LIMIT 1";

        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement pstmt = conn.prepareStatement(query);
             ResultSet rs = pstmt.executeQuery()) {

            if (rs.next()) {
                Gson gson = new Gson();

                // Restaurăm inamicii
                String enemiesMap1Json = rs.getString("enemies_map1");
                String enemiesMap3Json = rs.getString("enemies_map3");
                Type type = new TypeToken<ArrayList<String>>() {}.getType();
                ArrayList<String> enemiesData1 = gson.fromJson(enemiesMap1Json, type);
                ArrayList<String> enemiesData3 = gson.fromJson(enemiesMap3Json, type);
                ArrayList<Enemy> enemiesMap1 = Enemy.convertDataToEnemies(enemiesData1);
                ArrayList<Enemy> enemiesMap3 = Enemy.convertDataToEnemies(enemiesData3);

                // Restaurăm KeyCard
                boolean keycardCollected = gson.fromJson(rs.getString("keycard_state"), Boolean.class);
                int keycardX = rs.getInt("keycard_x");
                int keycardY = rs.getInt("keycard_y");

// Creăm keyCard-ul cu poziția originală
                Game.keyCard = new KeyCard(keycardX, keycardY);

                Game.keyCard.setCollected(keycardCollected); // Setăm starea colectată dacă a fost colectat
                Game.player.keyCardSetActive(keycardCollected); // Sincronizăm starea în Player


                // Restaurăm Imunosupressors
                String imunosMap1Json = rs.getString("imunosuppressors_map1");
                String imunosMap3Json = rs.getString("imunosuppressors_map3");

                ArrayList<Imunosupressor> imunosMap1 = new ArrayList<>();
                ArrayList<Map<String, Object>> imunosData1 = gson.fromJson(imunosMap1Json, new TypeToken<Arr
                for (Map<String, Object> data : imunosData1) {
                    int x = ((Double) data.get("x")).intValue();
                    int y = ((Double) data.get("y")).intValue();
                    boolean collected = (Boolean) data.get("collected");
                    Imunosupressor imunos = new Imunosupressor(x, y);
                    if (collected) imunos.setCollected();
                    imunosMap1.add(imunos);
                }

                ArrayList<Imunosupressor> imunosMap3 = new ArrayList<>();
                ArrayList<Map<String, Object>> imunosData3 = gson.fromJson(imunosMap3Json, new TypeToken<Arr
                for (Map<String, Object> data : imunosData3) {
                    int x = ((Double) data.get("x")).intValue();
                    int y = ((Double) data.get("y")).intValue();
                    boolean collected = (Boolean) data.get("collected");
                    Imunosupressor imunos = new Imunosupressor(x, y);
                    if (collected) imunos.setCollected();
                    imunosMap3.add(imunos);
                }
                int collectedImunosCount = rs.getInt("imunosuppressors_counter");
                //Game.player.Imunosupressor_Count = collectedImunosCount;
```

```
            // Setăm starea restaurată
            Game.setEnemies1(enemiesMap1);
            Game.setEnemies3(enemiesMap3);
            Game.imunosupressorsMap1 = imunosMap1;
            Game.imunosupressorsMap3 = imunosMap3;

            Game.player.Imunosupressor_Count = collectedImunosCount;

            return new GameState(
                    rs.getString("player_name"),
                    rs.getInt("score"),
                    rs.getInt("timeScore"),
                    rs.getString("level"),
                    rs.getInt("health"),
                    rs.getBoolean("game_status"),
                    rs.getDouble("player_x"),
                    rs.getDouble("player_y"),
                    rs.getInt("current_map"),
                    enemiesMap1,
                    enemiesMap3
            );
        }
    } catch (SQLException e) {
        System.out.println("Error loading game state: " + e.getMessage());
    }
    return null;
}
```

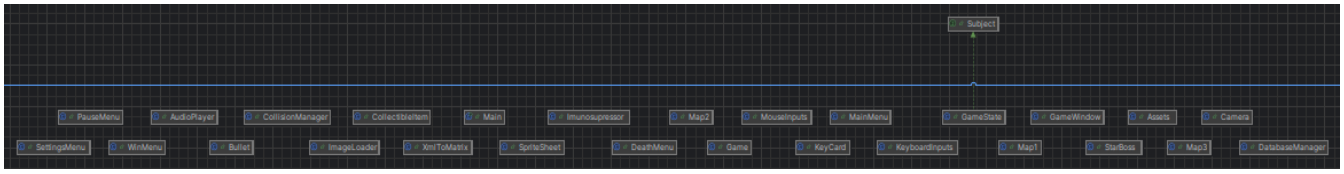
❖ **Reprezentarea pachetului Entities in diagrama UML**



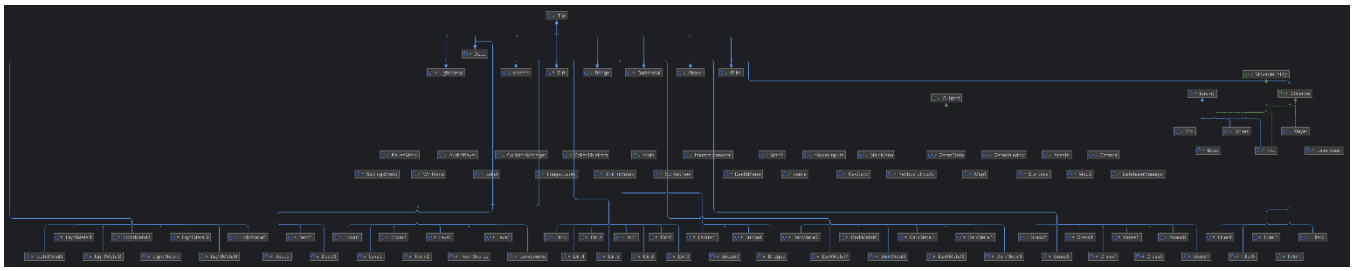
- **Reprezentarea Tile-urilor in diagrama UML**




- **Celelalte Clase din diagrama UML**




- **Diagrama UML completa si simplificata**



❖ **Diagrama UML completa cu campuri, constructori, metode, proprietati si clase interne.**

**Anexe (SpriteSheet and textures):**

Q
Shift

- **Limbaje de programare:** Java
- **Tool-uri suplimentare:** Tiled, Photoshop